Robust Cable Models from Complex Neuronal Morphologies

Jordan M. R. Fox[1,2*], Brian J. Fischer[3], William DeBello[4], Jose L. Peña[1]

[1]Dominick P. Purpura Department of Neuroscience, Albert Einstein College of Medicine.
[2]Zuckerman Institute, Columbia University.
[3]Department of Mathematics, Seattle University.
[4]Center for Neuroscience, University of California, Davis.

*Corresponding author(s). E-mail(s): jordanmrfox@gmail.com;

Abstract

We present a free and open-source, semi-automated, topologically robust pipeline for fitting cable models to 3D surface mesh morphology data of neuronal membranes, particularly suited to structures with complex shapes and topological holes. The motivation for this work is the discovery of morphologically complex neuronal spines on the auditory space-specific neurons of the barn owl, dubbed "toric spines", notable for their high curvature, branching density, and topological holes. Multicompartmental simulation software requires morphology to be represented as cable models, yet existing software tools for fitting cable models to complex 3D surface meshes generally assume acyclic topology, and thus did not work well for toric spines. We present the Mesh and Skeleton Cable Fitting (`MASCAF`) pipeline and software, which fits a cable model to a closed 3D surface mesh using mean-curvature flow skeletonization, and automatically handles cyclic topology. In this paper, we demonstrate how `MASCAF` is applied to fit cable models, how loop connections can be reconstructed in multicompartmental simulation software, and how the results can be validated using geometry and simulator-based methods. While non-tree morphologies such as toric spines are neuroanatomically special, our software pipeline provides a cable-model fitting approach for surface mesh data that is topologically robust, deterministic, open-source, and applicable to general morphologies, thereby closing a crucial gap between neuronal imaging and high-resolution simulation.

Keywords: neuron cable model, neuronal morphology, SWC, skeletonization, toric spines, mean curvature flow, multicompartmental simulation, surface mesh

1 Introduction

1.1 From Imaging Data to Multicompartmental Simulation

Neuronal morphology plays a fundamental role in shaping electrical signaling, synaptic integration, and neural computation (Rall 1962; Mainen and Sejnowski 1996; London and Häusser 2005). Since the foundational work of Rall and others on cable theory (Rall 1959), it has been well established that dendritic geometry strongly influences input resistance, voltage attenuation, temporal filtering, and nonlinear integration (Agmon-Snir et al. 1998; Stuart et al. 2016; Koch 1999). Moreover, advances in imaging technologies, particularly serial block-face scanning electron microscopy (SBEM) (Denk and Horstmann 2004) and stimulated emission depletion (STED) microscopy (Hell and Wichmann 1994), now allow reconstruction of neuronal structures at nanometer-scale resolution.

While imaging data and mesh representations preserve detailed geometric features, they are not directly compatible with modern multicompartmental simulators such as `NEURON` (Carnevale and Hines 2006), `GENESIS` (Bower and Beeman 1998), `Arbor` (Abi Akar et al. 2019), `MOOSE` (Dudley et al. 2022), `Brian 2` (Stimberg et al. 2019), `EDEN` (Panagiotou et al. 2022), and `Jaxley` (Deistler et al. 2025), which can provide valuable insights into the relationship between neuronal morphology and function. These simulators, using the well-established cable equations (Rall 1959; Carnevale and Hines 2006), represent neurons as graphs of isopotential compartments, with morphology typically specified by a cable model composed of connected cylindrical or frustum-shaped segments. In such frameworks, realistic morphological specification is important for accurate simulation results.

Fitting cable models, either directly to imaging data or to surface mesh data derived from imaging, is a general challenge (Wang et al. 2020; Liu et al. 2016; Peng et al. 2014). A variety of software tools have been developed for these tasks (Oxford Instruments; Peng et al. 2014; Sato et al. 2000; Acciai et al. 2016), but naturally, some approaches have limitations that make them inappropriate for particularly complex topologies. In this work, we present a semi-automated, topologically robust pipeline for fitting cable models to 3D surface mesh data, without restricting them to tree-like structures.

The central calculation performed by multicompartmental simulators like `NEURON` and `Arbor` is the construction and integration of the cable equations, a system of first-order nonlinear ordinary differential equations that describe changes in membrane potential for the set of isopotential compartments. Equation 1 shows the ODE for a single compartment μ connected to other compartments ν, describing the time evolution of membrane potential V_μ. Specific membrane capacitance of compartment μ, denoted c_μ, has units of capacitance per area. The term i_μ has units of electrical current per area and accounts for membrane mechanisms specific to compartment μ, such as passive currents and voltage-gated ion channels. External current injection into compartment μ is denoted I_μ^{ext}, and A_μ is its surface area. The sum in the last term accounts for current contributions from connected compartments ν, with $g_{\mu,\nu}$ denoting the conductance per unit area of the connection, which determines the resistive coupling between adjacent compartments and depends on the axial resistivity and the geometry of compartments μ and ν.

$$c_\mu \frac{dV_\mu}{dt} = i_\mu + \frac{I_\mu^{\text{ext}}}{A_\mu} + \sum_\nu g_{\mu,\nu}(V_\nu - V_\mu) \qquad (1)$$

The geometry and connectivity of the cable model strongly influence the cable equations and, thus, the membrane potential produced by the simulator. The goal of `MASCAF` is to provide more realistic cable models for complex morphologies and, therefore, improve the fidelity and quality of multicompartmental simulations.

The `MASCAF` pipeline assumes 3D surface mesh data as input (a triangle mesh of 3D vertices, edges, and faces, e.g., the Wavefront OBJ format (Wavefront Technologies 1990)). For morphologies derived from imaging data, this may require an intermediate membrane-tracing step, typically tracing the neuronal membrane on each slice and converting the result to a triangle mesh. As such, combining `MASCAF` with a membrane-tracing process to produce surface meshes constitutes a complete pipeline from imaging data to multicompartmental simulation. Many software tools exist for semi- or fully automated tracing of cell features in imaging data (Acciai et al. 2016); see Appendix C for a brief description of related software tools.

Cable models typically encode neuronal morphology as an *arborescence*: in graph theory, a directed, rooted, acyclic tree. SWC (Mehta et al. 2023) is a general-purpose cable model format widely used in neuroscience; `MASCAF` cable models are saved in this format. An SWC model describes an arborescence as a list of vertices, each with the following parameters:

- unique index (integer)
- tag (integer)
- spatial coordinates x, y, z (float)
- local radius (float)
- parent index (integer)

In a valid SWC model, every non-root node must have exactly one parent, implicitly enforcing acyclicity. `MASCAF` handles cyclic morphologies by

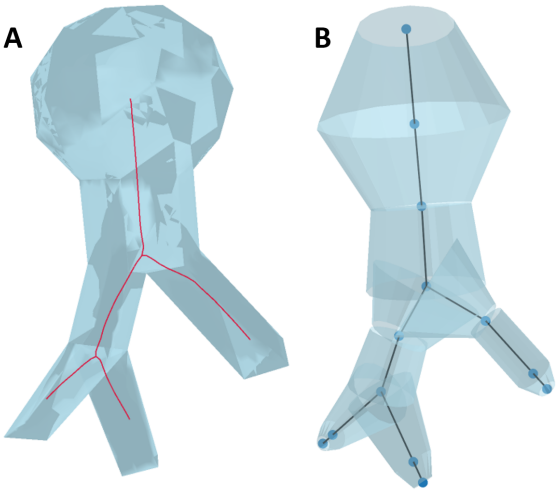


Fig. 1: A simple example of a 3D surface mesh with skeleton overlaid in red (A), and the corresponding cable model created using `MASCAF` (B). The cable model shows vertices (dark blue) and edges (black). The surface of the cable model is a union of frustum-shaped segments around each edge, determined by adjacent vertex radii.

breaking cycles at export and writing reconnection directives in the SWC header, which are used to restore loops in the simulator (Section 2.5).

Terminology of 3D geometry. Throughout this paper, a *skeleton* denotes a one-dimensional midline representation of the mesh volume. A skeleton may be represented by a structured point set, line set, or 3D graph. A *morphology graph* is a 3D graph whose vertices carry three-dimensional coordinates; it is a downsampled and optimized version of the skeleton that serves as the basis for the cable model. A *cable model* is a 3D graph whose vertices also carry local radius values; it is the discretized morphology used in multicompartmental simulation (and, when represented as a tree, may be written in SWC format). `MASCAF` computes a skeleton from a surface mesh, then constructs the cable model from the mesh and skeleton. Figure 1 (A) shows a simple example of a surface mesh (blue) with an overlaid skeleton (red), and (B) the corresponding cable model with vertices (dark blue), edges (black), and frustum-shaped segments deduced from radii (light blue).

1.2 Motivation: Toric Spines in the Barn Owl Auditory Pathway

The development of the `MASCAF` pipeline is motivated by the discovery of uniquely shaped dendritic protrusions, termed toric spines, on space-specific neurons (SSNs) in the barn owl midbrain (Sanculi et al. 2020). SSNs play a central role in the encoding of sound source location (Spezio and Takahashi 2003; Keller and Takahashi 2005; Singheiser et al. 2012); these neurons compute auditory spatial receptive fields via integration of synaptic inputs encoding sound frequency, interaural time difference (ITD), and interaural level difference (ILD) (Peña and Konishi 2001, 2002; Fischer et al. 2009). They are found in the external nucleus of the inferior colliculus (ICx) and arranged topographically to form a map of auditory space (Knudsen and Konishi 1978). The auditory space map in ICx is relayed to the optic tectum (OT) where it aligns and integrates with a visual space map (Knudsen 1982; Knudsen and Brainard 1991). This bimodal map is used to guide orienting movements during nocturnal hunting (Payne 1971; Konishi 1973; Knudsen and Konishi 1979; Moiseff and Konishi 1981; Peña and DeBello 2010).

The ICx is also a major site of experience-dependent plasticity (Knudsen 2002). Young owls raised wearing prismatic spectacles that optically displace the visual field develop adaptive shifts in SSN auditory tuning, which restore bimodal map alignment and behavioral integrity (Knudsen and Knudsen 1989; DeBello et al. 2001; Brainard and Knudsen 1993; Knudsen and Brainard 1991; Knudsen 2002). Manipulations of acoustic experience produce similar effects specific for ITD-frequency combinations (Gold and Knudsen 2000). In both paradigms, auditory tuning in the structure that provides auditory input to ICx, the ICcls, is not altered. Thus, rearrangement of the axonal projections to toric spines and SSNs represents a prime locus for microanatomical plasticity.

Toric spine morphology and connectivity are highly unusual. Compared with typical dendritic spines, toric spines exhibit overall large size, wide necks, a lack of distinct spine heads, and a narrow, tubular geometry (Hering and Sheng 2001; Nimchinsky et al. 2002). Most distinctively, they can exhibit topological holes: tunnels through the

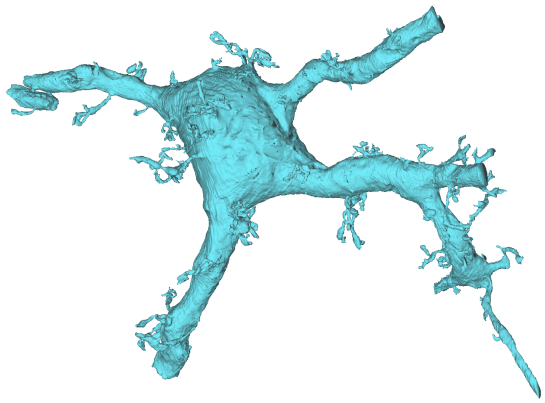


Fig. 2: Surface mesh reconstruction of an auditory space-specific neuron in the barn owl midbrain, compiled from electron microscopy data. Large dendritic processes extend out of the EM volume ($75 \times 45 \times 45\ \mu$m). Toric spines are visible as thorny structures covering the surfaces of the dendrites and soma.

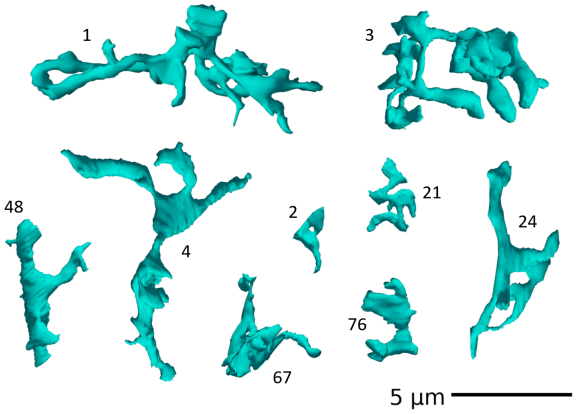


Fig. 3: Surface mesh reconstructions of nine selected toric spines from the barn owl ICx neuron shown in Figure 2. Each spine exhibits complex morphology including multiple necks, high curvature, and topological holes.

membrane that increase the genus of the closed surface mesh. These self-connections represent two distinct ultrastructures: cytoplasmic continuities which must be electrically contiguous, or junctions, which could be electrically contiguous depending on the presence/absence of connexin proteins that form gap junctions. This diversity of electrical properties is likely to powerfully shape the computations performed by SSNs. Highlighting this importance, a single toric spine can receive many synapses from many axons; 49 synapses from as many as 11 axons were observed (Sanculi et al. 2020). Thus, they are structured to act as integrators. A rigorous examination of the relationship between toric spine morphology and synaptic integration is a long-term goal of this work.

On the SSN chosen for high-resolution surface reconstruction from SBEM data, shown in Figure 2, 76 spines were identified, all classified as toric. The toric spines on this cell were observed to have anywhere from zero to nine holes, as well as from one to three necks per spine. Of the 76 toric spines observed, 9 were selected for detailed reconstruction and simulation. These surface meshes are from the reconstruction reported by Sanculi et al. (Sanculi et al. 2020); they are shown in Figure 3 and are publicly available at https://github.com/jmrfox/mascaf/tree/main/data/mesh/processed.

Software tools for neuron modeling often sensibly assume an arborescent structure (Oxford Instruments; Peng et al. 2014; Sato et al. 2000), but the holes of toric spines violate that assumption. The simulators `NEURON` and `Arbor` conveniently allow the establishment of direct self-connections via a gap junction-like mechanism (Carnevale and Hines 2006; Abi Akar et al. 2019), and, therefore, the core modeling challenge is in fitting a valid cable model to the surface mesh. Although development of `MASCAF` was motivated by the challenges presented by toric spine morphologies, which differ from common whole-neuron morphologies in overall scale and geometry, `MASCAF` can be applied similarly to any valid 3D surface mesh, as demonstrated in Figures 1, 9, and 10.

Toward the development of a pipeline for preparing toric spine morphologies for multicompartmental simulation, we identified several key design requirements. The pipeline must robustly compute one-dimensional skeletons from three-dimensional surface meshes, including geometries with high curvature, complex branching, and topological cycles. Structural cycles should generally be retained in the skeleton and the fitted cable model. The cable-model cycle count need not strictly equal the mesh genus, and it may be optimal in some cases to ignore topological holes that do not meaningfully change model structure. The resulting cable model must be exported

in SWC or an equivalent format suitable for direct construction of multicompartmental models. To maximize accessibility and reproducibility, the pipeline should operate directly on the surface mesh without requiring additional geometric inputs, rely primarily on deterministic geometric algorithms rather than global optimization or learned models, and be implemented entirely using free and open-source software.

Although the pipeline emphasizes geometric and topological fidelity, its ultimate purpose is to produce cable models for electrophysiological simulation rather than geometric reconstruction. The cable model serves as the morphological substrate from which multicompartmental simulators derive the electrical properties of the structure. Consequently, throughout this work we prioritize the accuracy of the resulting electrophysiological simulations over purely geometric or visual fidelity.

1.3 Contributions

The main contribution of this paper is the Mesh and Skeleton Cable Fitting (`MASCAF`) software (Fox 2026), which fits a cable model to a 3D surface mesh. We explain how the CGAL Triangulated Surface Mesh Skeletonization (`CGAL-TSMS`) algorithm is applied to provide a robust, computationally efficient approach for skeletonizing complex meshes, including those with cycles. Section 2 details methods used for skeletonization, graph optimization, and cable model fitting. In Section 3, we describe validation of our methods in geometric and simulation contexts. Section 4 notes limitations and opportunities for future work. Appendices provide a pipeline flowchart (A), algorithms (B), and a brief description of related software tools (C).

By enabling the conversion of high-resolution mesh reconstructions into multicompartmental simulation models without reliance on proprietary software, this pipeline lowers technical and financial barriers to morphology-driven computational neuroscience.

2 Methods

This section describes steps of the `MASCAF` pipeline for fitting a cable model to a 3D surface mesh. A flowchart of the `MASCAF` pipeline is shown in Appendix A. Mesh operations described in this section use the `trimesh` Python package (Dawson-Haggerty et al. 2019), and graph operations use `networkx` (Hagberg et al. 2008).

2.1 Input Data & Mesh Preprocessing

Mesh data is eligible for the `CGAL-TSMS` algorithm if and only if it is a single component, has no self-intersections and no boundaries (i.e., is "watertight"). If any of these prerequisites are not met, the operation will return an error before running. Meshes derived from imaging or contour data are made watertight either by standard mesh hole-filling routines or by alpha-wrapping before skeletonization (Alliez et al. 2026).

The `MASCAF` Python code offers two mesh preprocessing operations using `CGAL`: repair (Ohori 2026), and simplification (Cacciola et al. 2026). If the original mesh has a higher resolution than is needed for skeletonization, one may apply the simplification operation to reduce the number of vertices and edges. This will speed up the subsequent skeletonization operation, which scales with the number of mesh vertices. In our case, some meshes converted directly from contour data had several hundred thousand edges, and simplifying them to a few thousand edges greatly sped up operations without sacrificing fidelity.

2.2 Mean Curvature Flow Skeletonization via `CGAL-TSMS`

The central algorithm in `CGAL-TSMS` is mean-curvature flow skeletonization (MCFS), which iteratively displaces vertices according to local curvature until the mesh collapses to a one-dimensional skeleton (a curve for genus-0 meshes, or a graph that may contain cycles). `CGAL-TSMS` can be applied within the `MASCAF` Python code, in the `CGALLab` program (GUI), or programmatically in C++ code using the `CGAL` library (The CGAL Project 2026; Gao et al. 2026). A thorough description and analysis of MCFS can be found in the original paper (Tagliasacchi et al. 2012). A brief and application-focused description of the MCFS process is as follows.

2.2.1 Mathematical Foundation

Mean curvature flow skeletonization (MCFS) is a dynamical process applied to the closed 2D surface that contracts it toward a 1D midline. Consider a map $\mathbf{p} : U \subset \mathbb{R}^2 \to \mathbb{R}^3$ where $\mathbf{p}(u, v) = (x(u, v), y(u, v), z(u, v))$ and U is a closed domain in the u, v plane. We denote our closed 2D surface $S \equiv \{\mathbf{p}(u, v)|u, v \in U\}$, which encloses a 3D volume Ω, so $S = \partial\Omega$. Under MCFS, the time rate of change of the surface is equal to the local inward normal $-\mathbf{n}$ (where $\mathbf{n}$ is the outward unit normal, so $-\mathbf{n}$ points *into* the volume Ω) scaled by the local mean curvature $H = (k_1 + k_2)/2$, where k_1 and k_2 are the principal curvatures.

$$\dot{S} = -H\mathbf{n} \tag{2}$$

In practice, however, the vector $H\mathbf{n}$ is challenging to compute. Instead, MCFS invokes the Laplace-Beltrami operator on S, Δ_S, which is a generalization of the standard Laplacian operator to curved subspaces.

$$H\mathbf{n} = \Delta_S\mathbf{p} \tag{3}$$

The skeleton solution $\mathbf{p}^*$ is interpreted as having infinitesimal volume and surface area, and so mean curvature flow halts at that point.

$$\Delta_S\mathbf{p}^* = 0 \tag{4}$$

2.2.2 Discretized Mean Curvature Flow Skeletonization

In the discrete representation, S is approximated by a 2D surface mesh M_S, which is defined by vertices, edges, and triangular faces. Similarly, we denote the internal volume M_Ω. The mesh has vertices $V = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n]$, the locations of which constitute a discrete approximation of the function $\mathbf{p}$. The `CGAL-TSMS` algorithm updates vertex positions with an implicit Euler step, $V^t \to V^{t+1}$ until total change in V is below a convergence threshold. Further details about the solution process are discussed in the original publication (Tagliasacchi et al. 2012).

To understand the parameterization of MCFS relevant to `MASCAF`, we can consider the total "energy" for a configuration V at time t,

$$\begin{aligned} E =& \|w_L L V^{t+1}\|^2 + w_H \sum_i \|\mathbf{v}_i^{t+1} \\ & - \mathbf{v}_i^t\|^2 + w_M \sum_i \|\mathbf{v}_i^{t+1} - \mu(\mathbf{v}_i^t)\|^2, \end{aligned} \tag{5}$$

which is consequently minimized during MCFS. Here, L is the discrete Laplace-Beltrami operator, w_L, w_H, and w_M are weight parameters, and $\mu(\cdot)$ is a map from a vertex to its nearest Voronoi pole. Voronoi poles constitute a sparse approximation of the mesh's midline in the form of a point cloud; they are useful here in guiding V toward the midline, but do not constitute a solution to MCFS on their own. The three weights are constrained by a partition-of-unity property; only w_H and w_M must be set by the user, and w_L is computed internally. For each term in Equation 5, a larger corresponding weight value emphasizes that term's influence in the solution: a larger w_H leads to smaller changes per time step, and a larger w_M produces vertices closer to Voronoi poles.

The `CGAL-TSMS` C++ implementation has the following exposed parameters.

- `is_medially_centered` (boolean): Is $w_M > 0$?
- `quality_speed_tradeoff` (float): w_H
- `medially_centered_speed_tradeoff` (float): w_M

In `MASCAF`, `is_medially_centered` will generally be `true` because $w_M > 0$ provides a solution with vertices nearer to the mesh's midline. The values of w_H and w_M affect skeleton quality, smoothness, branching density, and compute cost, and we cannot provide a single parameterization that guarantees high-quality skeletons in general. Optimal balancing of parameters ultimately depends on features of the input data. For example, larger w_H can simultaneously increase skeleton detail and branching density, so an optimal value results in a balance of those effects. On the other hand, fitting of significant topological cycles is observed to be very robust under reasonable parameterizations. Suboptimal skeleton results can be optimized within `MASCAF` using methods discussed in Sections 2.3.2 and 2.3.3. For all our toric spine meshes, skeletons were generated with $w_H = 0.5$ and $w_M = 5$.

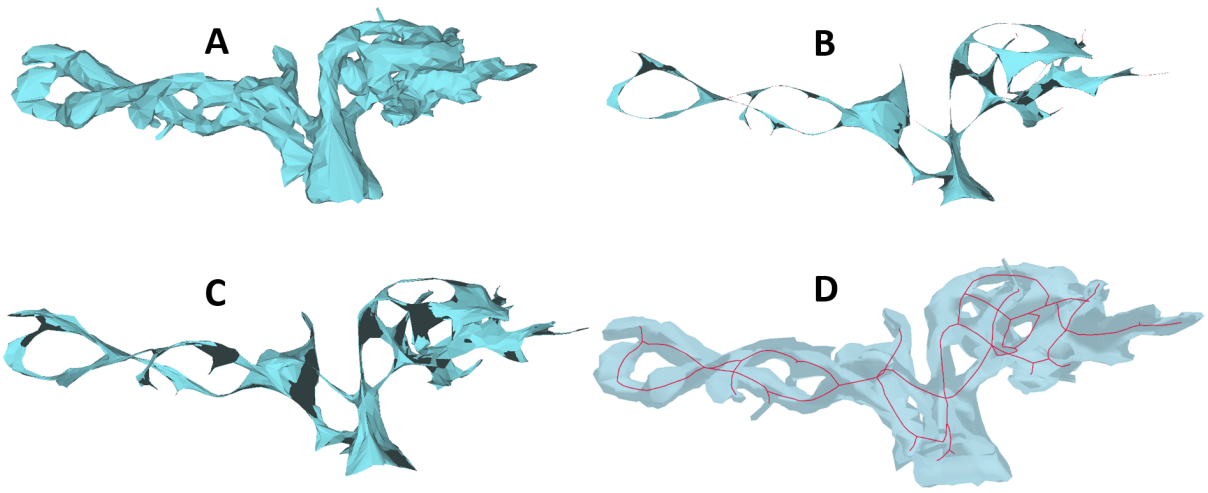


Fig. 4: Mean curvature flow skeletonization via `CGAL-TSMS` provides a reliable midline calculation. The original toric spine mesh (A) is iteratively thinned (B, C) to produce a 1-dimensional skeleton (D).

2.2.3 Skeleton Data: Polylines and the Skeleton Graph

The skeleton Σ computed by `CGAL-TSMS` is specified in standard *polylines* format and consists of a set of m ordered lists of 3D points:

$$\begin{aligned}\Sigma &= \left\{(x_1^i, y_1^i, z_1^i, x_2^i, y_2^i, z_2^i, \ldots, x_{N_i}^i, y_{N_i}^i, z_{N_i}^i)\right\}_{i=1}^{m} \\ &= \left\{(\mathbf{x}_1^i, \mathbf{x}_2^i, \ldots, \mathbf{x}_{N_i}^i)\right\}_{i=1}^{m}\end{aligned} \tag{6}$$

where $\mathbf{x}_j^i = (x_j^i, y_j^i, z_j^i)$ is a single 3D point (the j-th point in polyline i).

The locations of branch points (degree 3 or more) and terminal points (degree 1) can be deduced from polyline data, which allows us to construct the *skeleton graph* G_Σ. Each of the first and last coordinates in a polyline, $\mathbf{x}_1^i$ and $\mathbf{x}_{N_i}^i$, must be either a branch point or a terminal point. If the coordinate of an endpoint matches an endpoint of a different polyline within a small tolerance ε (e.g., $\mathbf{x}_{N_1}^1 \approx \mathbf{x}_1^2$), then the two polylines are connected at that point, and so it marks a branch point. If an endpoint is unique, then it is a terminal point. Points with $j = 2, 3, \ldots, N_i - 1$ must have degree 2. Furthermore, because M_S is a single component, G_Σ must be a single component. This is sufficient information to construct the skeleton graph, which is an undirected graph G_Σ with vertices V_Σ, and edges describing the connectivity structure implicit in the ordering of Σ.

2.2.4 Skeleton Quality

For a general three-dimensional surface mesh, there is no unique optimal one-dimensional skeleton. The medial axis transform of a 3D enclosed volume is generally a composite of 2D sheets, 1D curves, and 0D junctions (Blum 1967; Siddiqi and Pizer 2008). Skeleton quality must instead be judged with respect to the intended application. In `MASCAF`, a suitable skeleton should traverse the full extent of the enclosed volume, remain near its geometric midline, preserve the relevant topology, and contain sufficient branches to represent the morphology without introducing unnecessary geometric complexity. The broader implications of this non-uniqueness are discussed in Section 4.1.

The skeleton produced by `CGAL-TSMS` may not satisfy all of these criteria for every parameterization. While choosing $w_M > 0$ encourages contraction toward the medial region of the mesh, individual skeleton points may remain displaced from the local midline, and in some cases may lie outside the closed surface. Small deviations are generally tolerated by the subsequent radius-fitting procedure. However, when a skeleton vertex lies entirely outside the enclosed volume, a meaningful local radius may not be recoverable, and fitting at that location can fail.

Branching density presents a separate challenge. A skeleton with too few branches may fail to represent distinct morphological processes, whereas excessive or spurious branches may overfill the original volume once radii are assigned. The latter error cannot be assessed completely from the unweighted skeleton alone, because its geometric effect depends on the fitted radii and on the resulting union of cable segments. After radius fitting, branching quality can therefore be evaluated through volumetric reconstruction measures, including overlap between the reconstructed cable volume and the original mesh volume. Before radii are available, branching density must be estimated from the spatial relationship between the skeleton and the mesh surface. `MASCAF` provides a graph-pruning operation that can remove short or otherwise spurious branches before the final cable model is constructed.

When the initial skeleton is insufficiently centered or contains excessive branching, the user may modify w_H and w_M and rerun `CGAL-TSMS`. In practice, these objectives can compete. Increasing w_H generally preserves more of the local surface geometry and can improve morphological coverage, but it may also increase the number of branches retained during contraction. Conversely, stronger contraction toward the medial region may simplify the skeleton while reducing its ability to follow fine morphological features. Consequently, it may be difficult to identify a single parameterization that produces both a well-centered skeleton and an appropriate branching density. In the following section, we introduce an optimization procedure that refines the positions of morphology-graph vertices after the initial graph basis has been constructed from the skeleton.

2.3 Cable Model Fitting

Construction of the cable model proceeds in stages: constructing the base morphology graph G_M from the skeleton graph G_Σ, optional basis optimization (pruning, snapping, and forcing), fitting local radii by measuring local distances to the mesh surface, and global surface-area normalization. A pseudocode description of the fitting algorithm is included in Appendix B.2.

2.3.1 Basis Construction: the Morphology Graph

The morphology graph $G_M(V_M, E_M)$ is an undirected graph that will form the spatial and structural basis for the final cable model. The vertices V_M are instantiated with position values from the skeleton graph G_Σ. Typical skeletons from `CGAL-TSMS` have a higher spatial density of points than is useful for a cable model, so we resample G_Σ when computing the initial G_M. We first identify a set of all terminal and branch vertices as $C(V_\Sigma) = \{\mathbf{v} \in V_\Sigma \mid \deg(\mathbf{v}) \neq 2\}$, then tabulate the set of N unique non-branching sections, $\{g_\Sigma^k\}_{k=1}^N$. A non-branching section is a maximal edge-connected subgraph of one of two types: (i) a path whose endpoints lie in $C(V_\Sigma)$ and whose interior vertices all have degree 2, or (ii) a cycle whose vertices all have degree 2. The skeleton graph G_Σ is decomposed into non-branching sections g_Σ^k by tracing all maximal paths emanating from vertices in $C(V_\Sigma)$ and then tracing any remaining cycles. This yields an edge-disjoint decomposition of the graph into maximal non-branching sections: $G_\Sigma = \bigcup_{k=1}^N g_\Sigma^k$. Given a global maximum edge length parameter ℓ, each g_Σ^k with arc-length $d(g_\Sigma^k)$ is resampled to have $\lceil d(g_\Sigma^k)/\ell \rceil + 1$ evenly spaced vertices, including the endpoints, to give a new non-branching section g_M^k. Finally, the morphology graph is instantiated as the union of all of the resampled sections, $G_M = \bigcup_{k=1}^N g_M^k$. A section shorter than the maximum edge length ℓ becomes a single edge. The morphology graph does not keep more than one edge between the same two vertices, so a cycle that then passes through only two vertices disappears. `MASCAF` warns when this downsampling loses a cycle, and the fit is still allowed. The user controls that choice with ℓ. Dropping the cycle can be useful when `CGAL-TSMS` produces a small cycle that should not be kept. This gives a robust process to compute basis nodes from the skeleton while maintaining global control over edge length simply via ℓ.

Basis optimization, optional refinement of V_M positions to improve the quality of the final cable model, proceeds in three phases: pruning of short terminal branches, vertex snapping, and vertex forcing. The latter two phases are described in the following two sections, and a pseudocode description of the basis optimization algorithm is given

in Appendix B.1. Since vertex snapping and forcing scale with the number of vertices targeted, we intentionally apply these operations after the resampling $G_\Sigma \to G_M$.

2.3.2 Basis Correction: Vertex Snapping

Although rare, Σ will sometimes contain points fully outside M_S. This may occur when using suboptimal parameterizations of `CGAL-TSMS`, and is more likely to occur near regions of high curvature. This can cause errors when fitting radii with `MASCAF`. Vertex snapping identifies vertices outside the mesh volume and moves them inside by updating their positions. Figure 5 illustrates the vertex snapping process.

Let $\sigma(M_S, \mathbf{v}_i)$ be the point on M_S nearest to $\mathbf{v}_i$, and define the unit direction

$$\hat{\mathbf{u}}_i = \frac{\sigma(M_S, \mathbf{v}_i) - \mathbf{v}_i}{\|\sigma(M_S, \mathbf{v}_i) - \mathbf{v}_i\|} \ . \tag{7}$$

Casting a ray from $\mathbf{v}_i$ along $\hat{\mathbf{u}}_i$ yields ordered intersection points $\{\mathbf{q}_j\}$ with faces of M_S. The snapped position is a fraction γ of the chord between the first two intersections,

$$\mathbf{v}_i' = (\gamma - 1)\mathbf{q}_1 + \gamma \mathbf{q}_2 \ , \tag{8}$$

where γ controls the placement along the chord ($\gamma \approx 1/4$), provided that the chord length $\|\mathbf{q}_2 - \mathbf{q}_1\|$ exceeds a small threshold relative to the mesh size.

Although using $\gamma = 1/2$ would very often place $\mathbf{v}_i'$ nearer to the mesh midline, this can also lead to extreme angles between the edges of G_M adjacent to $\mathbf{v}_i'$, thereby introducing artificial discrepancies between the tangents of G_M and those of M_S. This procedure is still inexpensive when only a few vertices lie outside the mesh, and is often sufficient to proceed with radius fitting. If higher midline accuracy is necessary, we continue to the vertex forcing phase of basis optimization.

2.3.3 Basis Refinement: Vertex Forcing

Vertex forcing uses an iterative method to move any vertices that are inside the mesh closer to the midline. Figure 6 illustrates one iteration. Terminal vertices are typically held fixed (and optionally branch vertices), so that connectivity anchors are

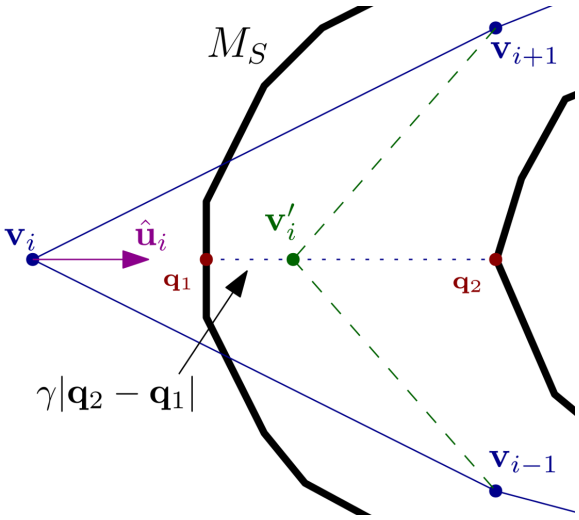


Fig. 5: Vertex snapping for a vertex $\mathbf{v}_i$ (blue) outside the mesh surface M_S (black). A ray in the direction of the unit vector $\hat{\mathbf{u}}_i$ (magenta) intersects the surface at ordered points $\mathbf{q}_1$ and $\mathbf{q}_2$ (red). The updated vertex $\mathbf{v}_i'$ (green) lies on the chord between them at fraction γ of the distance from $\mathbf{q}_1$ toward $\mathbf{q}_2$.

preserved while non-terminal vertices are refined. This process moves a vertex $\mathbf{v}_i$ along an iterative trajectory described by $\mathbf{v}_i^{t+1} = \mathbf{v}_i^t + \Delta\mathbf{v}_i^t$. The total force is a mixture of two contributions,

$$\mathbf{f} = h\left[(1 - \alpha_s)\mathbf{f}_c + \alpha_s \mathbf{f}_s\right] \tag{9}$$

where $\mathbf{f}_c$ is a centering force, $\mathbf{f}_s$ is a smoothing force, $h \geq 0$ controls the step scale, and α_s controls the smoothing fraction. To prevent a step from carrying a vertex outside the mesh, we limit the possible change at each iteration. We define the maximum step distance $f_{\max}$ to be a tunable constant fraction (default 1/2) of the distance from $\mathbf{v}_i^t$ to the nearest face of M_S in the direction of $\mathbf{f}$. The applied step is then

$$\Delta\mathbf{v}_i^t = \frac{\mathbf{f}}{\|\mathbf{f}\|} \min\big(\|\mathbf{f}\|,\, f_{\max}\big) \tag{10}$$

The purpose of the centering force $\mathbf{f}_c$ is to push the vertex away from the surface using ray-tracing. To compute the centering force on $\mathbf{v}_i^t$, we cast N_R rays from the vertex with unit directions $\{\hat{\mathbf{x}}_j\}_{j=1}^{N_R}$. By default, $N_R = 6$ with unit vectors aligned with the positive and negative Cartesian axes; denser

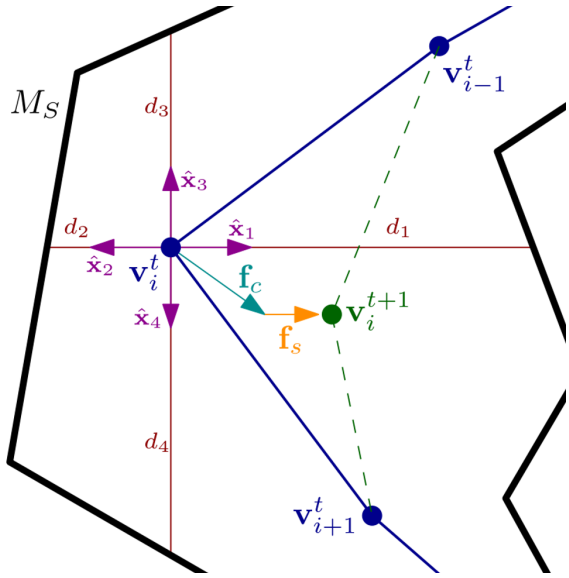


Fig. 6: One vertex-forcing iteration for an interior vertex $\mathbf{v}_i^t$ (blue), projected onto a 2D plane. Distances d_j (red lines) from $\mathbf{v}_i^t$ to the mesh M_S (black) along four ray directions given by unit vectors $\hat{\mathbf{x}}_j$ (magenta) determine the centering force $\mathbf{f}_c$ (cyan) according to Equations 11 and 12. The smoothing force $\mathbf{f}_s$ (orange) pulls $\mathbf{v}_i$ toward neighboring vertices. Note that the step scale h and smoothing fraction α_s are omitted from this figure for clarity. Vector directions and magnitudes are chosen purely for illustration.

sampling on the sphere (e.g., Fibonacci sampling) can be used when higher accuracy is desired, at greater computational cost. Along each ray, we compute the scalar distance d_j measured from $\mathbf{v}_i^t$ to the nearest intersection with a face of M_S. The centering force $\mathbf{f}_c$ is the weighted sum of the ray unit vectors,

$$\mathbf{f}_c = -d_{\min} \frac{\sum_{j=1}^{N_R} \hat{\mathbf{x}}_j w_j}{\sum_{j=1}^{N_R} w_j} , \tag{11}$$

kept at its weighted magnitude (not re-normalized to unit length), where the weight for direction $\hat{\mathbf{x}}_j$ is

$$w_j = (d_j + \epsilon)^{-p} \exp\left(-\beta \frac{d_j - d_{\min}}{d_{\min}}\right) . \tag{12}$$

At each iteration, we compute $d_{\min} = \min\{d_j\}_{j=1}^{N_R}$, the minimum distance from the vertex to M_S along the rays, which is then used to scale the force weights.

A weight w_j has two multiplicative contributions, one repulsive, and one localizing. The repulsive factor is $(d_j + \epsilon)^{-p}$, where ϵ is a small number to avoid division by zero, and p is a parameter controlling the shape of the repulsion. This factor is larger for small d_j, and the nonlinearity introduced by the exponent p (typically $p \in [1, 2]$) allows for small distance repulsions to be further emphasized over large ones. The exponential factor is for localizing; it allows for the centering force to be dominated by short-range repulsion, and effectively ignores long-range effects. The localizing term exponentially downweights contributions with $d_j > d_{\min}$. The parameter β controls the shape of localization, and is similar in effect to p. This weight design produces reliable local centering and provides the user with global control. Default values in `MASCAF` of $p = 1$ and $\beta = 2$ are adequate for general applications. Note that each term may be turned off by setting its corresponding parameter to zero.

The smoothing force $\mathbf{f}_s$, introduced to suppress jitter and maintain spatial coherence, points the vertex $\mathbf{v}_i^t$ toward the average of positions in the set of neighboring vertices $\mathcal{N}(\mathbf{v}_i^t)$:

$$\mathbf{f}_s = \frac{\sum_{k \in \mathcal{N}(\mathbf{v}_i^t)} \mathbf{v}_k^t}{|\mathcal{N}(\mathbf{v}_i^t)|} - \mathbf{v}_i^t. \tag{13}$$

While $\mathbf{f}_c$ approaches zero when $\mathbf{v}_i^t$ is centered (neglecting long-range forces), $\mathbf{f}_s$ does not. Therefore, the practical use of vertex forcing is not to run until $\|\mathbf{f}\|^2$ goes to zero, but rather to either run until $\|\mathbf{f}_c\|^2$ crosses below a threshold, or simply for a fixed number of iterations T. In practical applications of vertex forcing, we find that relatively small iteration counts ($T < 50$) produce satisfactory refinement.

2.3.4 Computing Radii

The local radius of each vertex in V_M is computed via the following process. Consider a target vertex $\mathbf{v}_i \in V_M$. For each adjacent edge e_{ij}, we define a cross-section plane through $\mathbf{v}_i$ with normal $\mathbf{e}_{ij}$. Where the plane intersects the mesh, a polygonal cross-section with area A is computed using standard mesh manipulation algorithms (`trimesh`

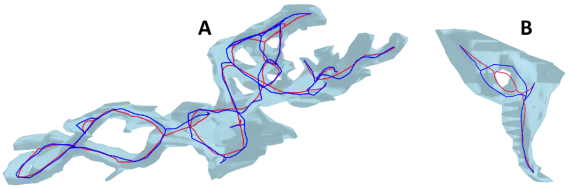


Fig. 7: Two toric spine meshes rendered with downsampled skeletons and optimized morphology basis graphs. Skeleton-graph vertices from `CGAL-TSMS` are shown in red, and morphology-graph vertices after `MASCAF` basis optimization are shown in dark blue. Note that, in (B), the initial skeleton is not fully contained within the mesh, while the optimized morphology graph is.

(Dawson-Haggerty et al. 2019)). We evaluated several different radius calculation strategies, including comparisons of surface-area-preserving and volume-preserving approaches. Although surface area plays a significant role in the cable equations, we found that a surface-area-preserving approach to radii calculations resulted in noisy radius distributions and larger geometric errors. The most accurate and practically robust strategy we found calculates the radius of a disk with the same area as the cross-sectional polygon; thus $r(A) = \sqrt{A/\pi}$. However, we acknowledge that surface area preservation may be an optimal strategy in some applications, and for that reason the option is included in the `MASCAF` code. This process is repeated for each edge e_{ij} connected to $\mathbf{v}_i$, producing a set of radii $\{r_j\}_{j=1}^{|\mathcal{N}(\mathbf{v}_i)|}$. Finally, the assigned radius r_i is a reduction of those, typically mean (the default) or median. Once every vertex in V_M has a radius, we treat G_M together with its frustum segment geometry as the cable model.

2.4 Fine Tuning: Surface Area Normalization & Terminal Extension

Compartment surface area has a significant effect on electrical simulations (Equation 1), augmenting or suppressing the effects of external currents. `MASCAF` therefore applies a final normalization step: a global scale multiplies all radii so that the sum of segment lateral surface areas matches the mesh surface area. Branch-point overlap, which motivates that scale, is estimated only when computing geometric errors (Section 3.1.1). Multicompartmental simulators typically use lateral membrane area in the cable equations, so normalization by default excludes terminal end caps (Carnevale and Hines 2006). Relative scales between radii within the same model are preserved, and thus relative compartment membrane areas are preserved under uniform specific membrane properties.

Lastly, `MASCAF` includes a convenient method to extend all terminal points (degree-1 vertices) of a cable model with an additional segment, scaled to the local radius. Due to medial centering, skeletons computed with `CGAL-TSMS` often feature branches that stop short of the end of the mesh. Depending on mesh geometry, this may lead to substantial geometric errors. For example, note how the skeleton of the branching geometry shown in Figure 1 (A) stops near the center of the bulk, but the cable model (B) has an additional terminal point at the top (and also ones on the lower branch terminals), added using terminal extension. The terminal-extension functionality adds new terminal points so the cable model extends beyond the skeleton, leading to reduced geometric errors, though it is not necessarily appropriate for all cases.

2.5 Simulator Integration

Since the SWC format is limited to describing acyclic graphs, cycles in the cable model must be resolved prior to export. `MASCAF` addresses this limitation by effectively breaking graph cycles and writing reconnection directives to the SWC file header. On each detected cycle $g_\circ^k \subset G_M$, one degree-2 vertex $\mathbf{v}$ is selected. This vertex is duplicated to create a clone $\mathbf{v}'$ with identical attributes, and one of the edges terminating on $\mathbf{v}$ is updated to instead terminate on $\mathbf{v}'$. The two duplicate vertices are distinct in the SWC specification and are not connected by an edge. This operation thus breaks the cycle while preserving geometric embedding. `MASCAF` adds metadata describing each broken cycle to the SWC file header, including the indices of $\mathbf{v}$ and $\mathbf{v}'$.

After initializing the morphology in simulation software, the original loop $g_\circ^k$ can be restored by adding a direct connection between $\mathbf{v}$ and $\mathbf{v}'$. In `Arbor`, for example, existing gap-junction functionality (normally used to connect two neurons)

can be repurposed to place a direct connection between the corresponding segments, exposing a conductance parameter for each connection (Abi Akar et al. 2019). A similar procedure can be implemented in `NEURON` (Carnevale and Hines 2006). We set the conductance of this connection (in siemens) to the reciprocal of axial resistance as a lumped electrical approximation of full cytoplasmic continuity (Carnevale and Hines 2006). As such, the `MASCAF` approach allows the modeler to recreate topological cycles at the simulation level while still using the common SWC format.

3 Results

This section presents selected results from `MASCAF` and covers geometric and simulation validation of the fitted cable models. As a representative example, we report validation metrics for toric spine #1 (TS1), fitted from a mesh with 2378 vertices. The original mesh, skeleton, and two cable model results for TS1 are shown in Figure 8. Before cycle-breaking, the cable model had 69 vertices, 75 edges, and 7 independent cycles; after cycle-breaking and SWC export, the arborescence has 76 vertices and 75 edges. Other attributes and error estimates are given in Table 1.

To illustrate the applicability of `MASCAF` to full neuronal morphology data, we include results for a human neuron, shown in Figure 9. The morphology mesh data for this human cortical interneuron (middle temporal gyrus), reconstructed by the Allen Institute, was provided by NeuroMorpho.org (Koch and Jones 2016; Tecuatl et al. 2024; Ascoli et al. 2007).

3.1 Geometric Validation

To evaluate geometric accuracy of results, we construct a frustum-based geometric proxy from the fitted cable model and compare its volume and lateral surface area to those of the original mesh M_S. Figure 10 shows `MASCAF` results for two synthetic meshes with tractable geometry: a straight cylinder and a torus. Consider the cable model graph $G_M(V_M, E_M)$ with edge lengths and vertex radii from the fitting pipeline. For each pair of vertices $\mathbf{v}_i, \mathbf{v}_j \in V_M$ that share an edge e_{ij} with length $|\mathbf{e}_{ij}|$, we construct a *segment* geometry, a frustum with radii r_i, r_j, which has volume

$$\frac{\pi}{3}|e_{ij}|(r_i^2 + r_j^2 + r_i r_j) \tag{14}$$

and lateral surface area (not including end caps)

$$\pi(r_i + r_j)\sqrt{|e_{ij}|^2 + (r_i - r_j)^2}. \tag{15}$$

The total volume of the cable model is estimated as the sum of all segment volumes. The total surface area used for comparison with the mesh is the sum of all segment lateral surface areas.

The `MASCAF` Python code includes a geometric validation system that compares the surface area and volume of the mesh to those of the cable model, with and without overlap subtraction (see Section 3.1.1). Mesh genus counts every hole. Very small holes that do not substantially change the morphology can be dropped when the mesh is simplified to a skeleton, and resampling can remove a short cycle, as described in Section 2.3.1. The cable model can therefore have fewer cycles than the genus. For toric spine #1, the mesh genus measured with `trimesh` is 9, but two of those holes are very small pinch points rather than structural tunnels, and the fitted cable model has 7 independent cycles. The `MASCAF` validation routine reports the measured mesh genus and the cable-model cycle count so they can be compared.

Table 1 reports geometric validation metrics and pipeline cost for the nine toric spines used in this study and for four other meshes: a human cortical interneuron and the synthetic cylinder, torus, and branching models. The toric-spine runs used 20 iterations of vertex forcing. The human neuron and the synthetic meshes used no vertex forcing because their skeletons were already accurate. Because only the toric-spine runs include vertex forcing, times should not be compared across the two blocks as if the forcing setting were the same. End-to-end wall-clock time was measured on a single workstation (Intel Core i9 CPU at 3.60 GHz, 8 cores) for the fitting runs underlying the table. All listed runs required peak RAM between 151.8 MB and 231.1 MB. Overall run time is governed mainly by skeleton size $|V_\Sigma|$, and vertex forcing is the most costly phase. `CGAL-TSMS` scales appreciably with the mesh vertex count $|V|$; elsewhere in `MASCAF`, $|V|$ enters mainly because the input surface must be retained in memory

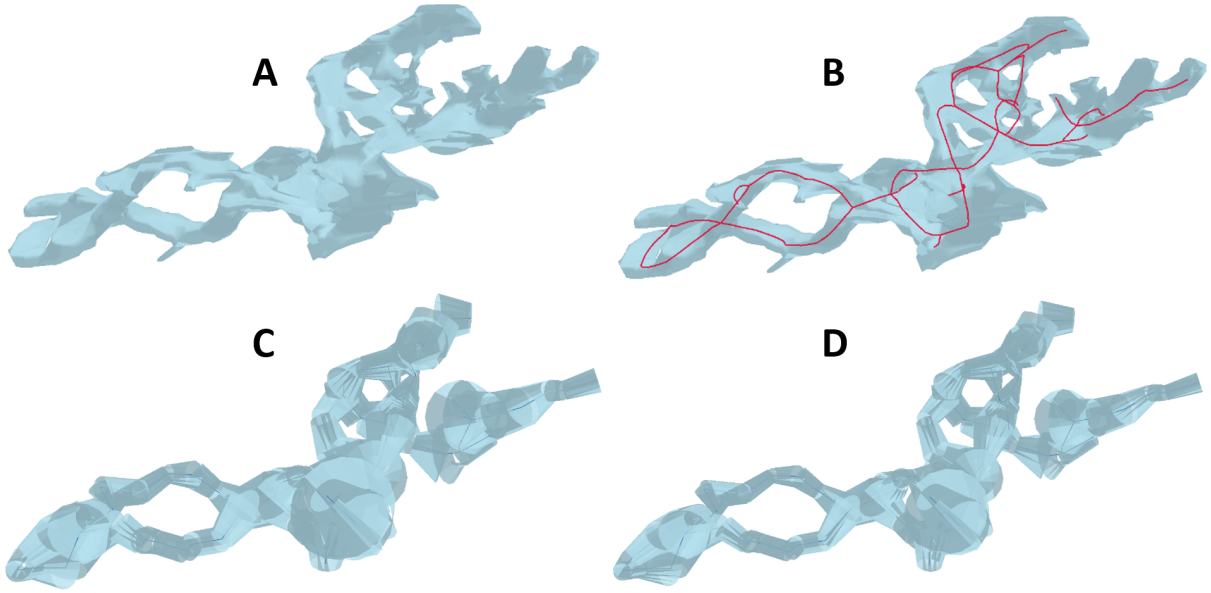


Fig. 8: Toric spine #1. (A) Original surface mesh. (B) Mesh (blue) with the skeleton (red). (C) Fitted cable model. (D) Final cable model after normalizing surface area to match that of the mesh. The fitted model (C) and area-scaled model (D) are different but visually quite similar, which suggests the radius calculation of Section 2.3.4 is accurate. Error estimates are listed in Table 1.

for geometric queries, whereas the cable fitting algorithm itself is largely independent of $|V|$. For analysis of the time demands of `CGAL-TSMS`, we refer readers to Tagliasacchi et al. (2012).

The values in Table 1 are from automated runs, with vertex forcing as described above. Lower errors are possible with more careful case-by-case tuning. The δ_r column is the primary geometric error metric: an estimate of radius error in percent, derived from volume and surface area errors,

$$\delta_r = \left[\sqrt{(V_{\text{cable}}/V_{\text{mesh}})^{1/3}(A_{\text{cable}}/A_{\text{mesh}})^{1/2}} - 1\right] \times 100. \tag{16}$$

3.1.1 Overlap estimation

A fundamental problem in geometric fitting of cable models arises from volumetric and surface-area overlap at vertices with more than two edges (branch nodes). At each vertex of degree greater than 2, there is a systematic positive overlap in volume and surface area of the connected segments. Depending on attributes of the morphology like branching density, curvature, and compartment shape, the errors introduced by these overlaps may or may not require attention. For many whole-neuron morphologies with thin neurite processes much longer than their radius, like the one shown in Figure 9, the contributions of segment overlaps to volume and surface area relative to the total may be negligibly small. However, if the mesh has wide compartments, high curvature, and high branching density, the overlaps can contribute substantially. In toric spines, the volume and lateral surface area estimated from segments of the cable model are consistently greater than those of the mesh because of overlaps. Summed segment volume and lateral surface area therefore include superfluous contributions where segments overlap at branch points. We estimate those contributions and subtract them when comparing the cable model to the mesh. Subtracting overlaps removes those contributions, making the geometric errors in Table 1 more interpretable.

To estimate overlaps, we use a cylinder-intersection model based on the Steinmetz solid: the shape of the intersection of two perpendicular cylinders of equal radius. Overlap for degree-2 vertices is assumed negligible: overlap on the inside of a bend is approximately balanced by missing geometry on the outside. For degree $d > 2$ vertices, each additional branch beyond degree 2 is

Table 1: Geometric validation and compute cost. Above the horizontal rule: nine toric spines; those runs used 20 iterations of vertex forcing. Below the rule: a human cortical interneuron (Figure 9) and three synthetic meshes (cylinder and torus in Figure 10, branching model in Figure 1); those runs used no vertex forcing. Terminal extension was used for the branching synthetic mesh and the human neuron. Recorded wall-clock time includes vertex forcing when it is used. ℓ: maximum edge length in the cable fitting algorithm. $|V|$: number of vertices in the input surface mesh. $|V_\Sigma|$: number of vertices in the skeleton graph G_Σ. $|V_M|$: number of vertices in the resampled morphology graph G_M after basis optimization. g: cyclomatic number of the cable model graph G_M (independent cycles). B: number of branch vertices in G_Σ. t: end-to-end `MASCAF` wall-clock time in seconds. δ_r: the primary geometric error metric, an estimate of radius error in percent, derived from volume and surface area errors.

Model	ℓ	$\lvert V\rvert$	$\lvert V_\Sigma\rvert$	$\lvert V_M\rvert$	g	B	t (s)	δ_r
TS1	155	2378	376	69	7	25	4.53	0.3
TS2	145	508	122	7	1	4	0.55	2.0
TS3	186	8008	714	70	10	47	15.80	1.9
TS4	212	3333	319	34	1	13	5.28	3.7
TS21	133	2017	251	21	1	11	1.89	2.9
TS24	189	3331	323	33	2	15	3.96	6.5
TS48	215	3335	255	19	0	7	2.34	4.1
TS67	156	1694	296	30	2	13	2.41	17.3
TS76	173	1509	191	16	0	7	1.20	0.4
human	1	3335	617	792	0	12	5.08	−5.2
cylinder	3	34	4	5	0	0	0.01	−0.9
torus	1	512	32	25	1	0	0.10	−0.3
branching	1	1417	137	13	0	2	0.03	−3.2

approximated as an equal-radius branch intersecting a straight cable at right angles. For volume, that reference geometry is half of an equal-radius Steinmetz solid. The intersection volume of two perpendicular cylinders of radius r is $\frac{16}{3}r^3$, so half of that intersection, per extra branch, contributes

$$\Delta V = \frac{8}{3}r^3. \tag{17}$$

Similarly, the buried lateral surface associated with the additional branch is approximately

$$\Delta A = 4r^2. \tag{18}$$

Thus, for a vertex of degree $d > 2$ and local radius r, the overlap corrections are

$$\begin{aligned} V_{\text{overlap}}(r,d) &= \frac{8}{3}(d-2)r^3, \\ A_{\text{overlap}}(r,d) &= 4(d-2)r^2. \end{aligned} \tag{19}$$

These are useful heuristics for estimating the true overlaps, which depend on various angles of intersection and non-cylindrical volumes. The scale factors 8/3 (volume) and 4 (area) are exposed as parameters in the `MASCAF` validation functions so that they can be adjusted if needed.

3.2 Simulation Validation

To verify the compatibility of generated cable models with multicompartmental simulation, reconstructed toric spine morphologies were imported into `Arbor` and simulated under standard alpha-synapse stimulation. The objective of these simulations was to confirm that morphologies produced by the `MASCAF` pipeline could be stably instantiated, discretized, and simulated without numerical or topological failure. Passive membrane parameters were assigned uniformly across compartments, and membrane potential probes were placed at every compartment to evaluate voltage propagation throughout the model.

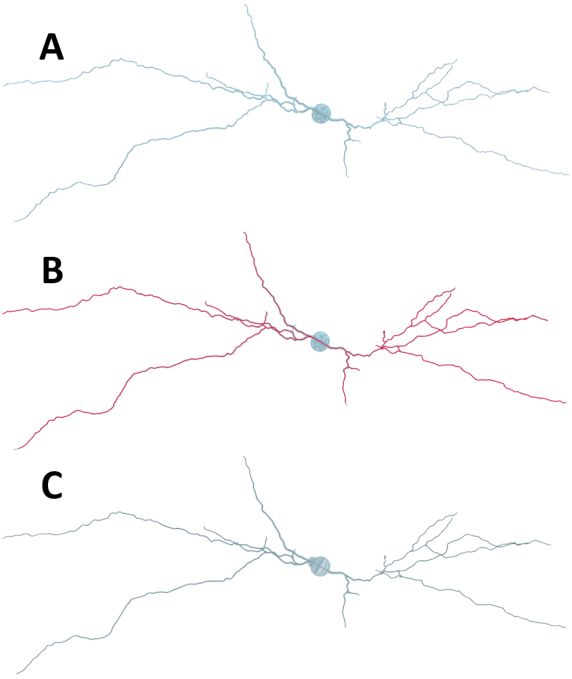


Fig. 9: Three steps illustrating `MASCAF` applied to the surface mesh of a human cortical interneuron from the middle temporal gyrus. (A) shows the neuron mesh, (B) shows the mesh (blue) with the skeleton (red), and (C) shows the cable model. The fitted cable model is similar to the original mesh. Geometric errors for this example are listed in Table 1.

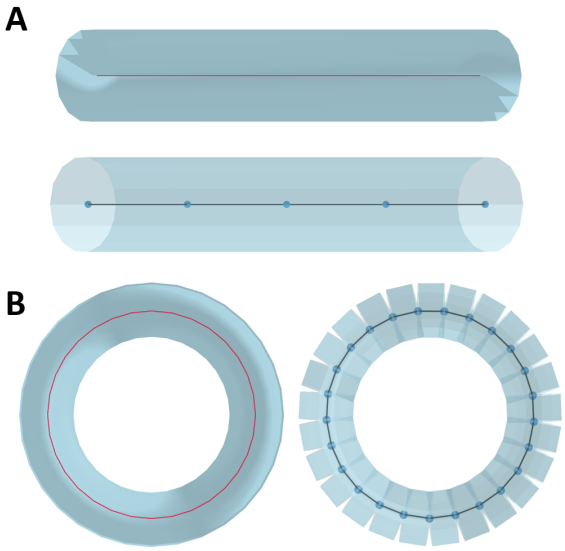


Fig. 10: Known-shape validation on synthetic meshes. (A) Cylinder: input surface mesh with skeleton (top) and fitted cable model (bottom). The cable model has 5 vertices and 4 compartments. Segment boundaries are hard to see because the segments are parallel and meet. (B) Torus: input surface mesh with skeleton (left) and fitted cable model (right). This cable model illustrates how the overlap of segments around a degree-2 vertex (see Section 3.1.1) tends to cancel with the opposing gap.

The resulting simulations exhibited stable electrotonic membrane dynamics and continuous voltage propagation across all tested morphologies, including cyclic regions of the toric spines. With neck coordinates retrieved from imaging data, we attached a large sink volume to the spine model at that point to allow for realistic electrotonic coupling to a dendritic compartment (Koch 1999; Stuart et al. 2016). A snapshot of one such simulation is shown in Figure 11. Locations of 25 active postsynaptic densities were retrieved from imaging data and recreated with alpha-synapse models (Carnevale and Hines 2006). No simulator instability, disconnected compartments, or topology-related discretization failures were observed during model initialization or time integration. Voltage responses followed the qualitative behavior expected from passive cable theory, with attenuation and temporal smoothing increasing as a function of path length and branching complexity (Rall 1959, 1962).

To further validate the cable models quantitatively, we measured peak membrane potential at every compartment as a function of geodesic distance from the stimulus site, under passive membrane parameters with a single alpha-synapse stimulus. Figure 12 shows the resulting attenuation profile for toric spine #1, the morphology of which is shown in Figure 8. Peak voltage decays monotonically with path distance, consistent with electrotonic attenuation predicted by passive cable theory (Rall 1959, 1962). Furthermore, the scatter of compartment peaks at similar path distances is representative of electrotonic spread over the branching and looped cable model.

These results demonstrate that cable models generated using `MASCAF` are compatible with modern multicompartmental simulators and produce

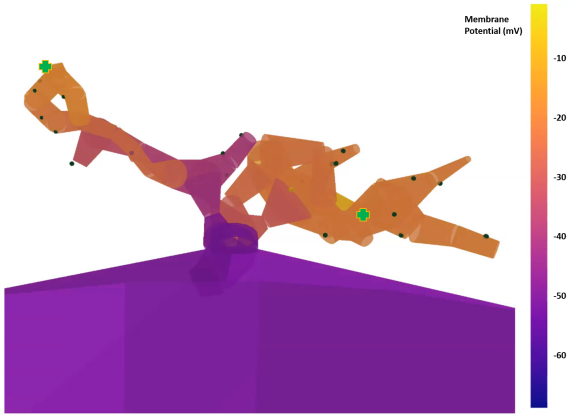


Fig. 11: Snapshot from an `Arbor` simulation using a cable model from `MASCAF`. The spine is attached to a large sink volume to allow for realistic electrotonic loading. Membrane potential (mV) is shown on the color scale. Synapse zones (25) are rendered on the surface as dark green dots. Two locations that received a presynaptic activation are marked with green plus signs (+); their changing conductances led to the visible local depolarization.

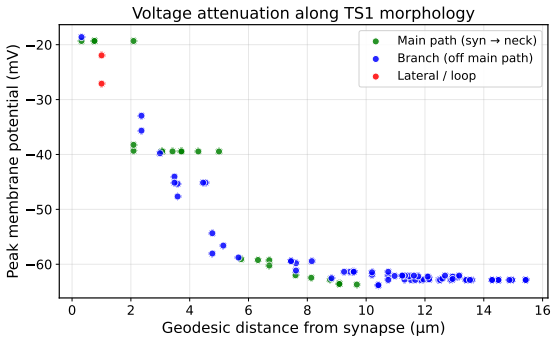


Fig. 12: Peak membrane potential vs. geodesic distance from a distal alpha-synapse stimulus site in a passive simulation of toric spine #1 using `Arbor`. Monotonic decay is consistent with electrotonic attenuation in a passive cable model.

electrophysiological behavior consistent with passive cable theory, supporting the geometric and topological metrics reported above.

4 Discussion

We present the Mesh and Skeleton Cable Fitting (`MASCAF`) pipeline for fitting cable models (e.g., SWC) to 3D surface meshes, particularly suited for topologically complex neuronal structures. Cable models, like the SWC format, are the most common morphology specification used in multicompartmental simulators like `NEURON` and `Arbor`, due to their straightforward use in cable-equation models (Ascoli et al. 2007; Mehta et al. 2023). Development of `MASCAF` was motivated by the notable discovery of a new kind of dendritic spine on auditory space-specific neurons in the barn owl's midbrain. Dubbed toric spines (Sanculi et al. 2020), they prominently feature topological holes, whose function is unknown. Cyclic topology of these morphologies breaks the assumption of tree-like neural structures (Oxford Instruments; Peng et al. 2014; Sato et al. 2000). We showed how the CGAL Triangulated Surface Mesh Skeletonization operation is used to compute midline skeletons for surface meshes via mean curvature flow. Structural cycles are retained; very small holes can be dropped in skeletonization, and the maximum edge length can remove a short cycle during resampling (Sections 2.3.1 and 3.1), so the cable-model cycle count need not equal the mesh genus. The `MASCAF` Python code (Fox 2026) uses the combined mesh and skeleton data to construct a cable model and write it to an SWC file, and also includes tools for quality optimization of the skeleton and of the morphology graph prior to radius fitting. For cyclic topologies, the output file is a valid arborescence, a rooted, directed tree, with cycle-closure directives carried in the file header; these can be read during the simulation phase to establish electrical connections, thereby preserving the real morphology in multicompartmental simulations.

4.1 Parameters, Limitations, & Opportunities

The most important global parameters of the `MASCAF` pipeline for cable model quality are w_H, the `quality_speed_tradeoff` in `CGAL-TSMS`, and ℓ, the maximum edge length in the cable fitting algorithm. As discussed in Subsections 2.2.2 and 2.2.4, w_H controls both the small-scale quality of the skeleton and its branching density. For small values of w_H, the skeleton may not follow the midline well, while for large values, the skeleton may include spurious branching. Spurious branching describes branches in the skeleton graph G_Σ that

have no corresponding volume in the mesh M_S. This is a problem because it can introduce unrealistic structure in the cable model, but simple volumetric validation metrics might give acceptably low errors. This is a limitation of the pipeline because, in general, the user might run `CGAL-TSMS` for several values of w_H and choose one based on visual accuracy, since we do not have an algorithmic metric for spurious branching. However, a robust oracle system could be developed to measure geometric properties of the mesh and suggest values for `CGAL-TSMS` parameters.

The maximum edge length ℓ presents a similar issue. If ℓ is too small, then the number of compartments becomes large, and thus the simulation cost grows. If ℓ is too large, then the model and simulation may not be a realistic representation of the biological system. A simple heuristic in the `MASCAF` Python code sets ℓ to a scalar multiple of the median of the shape diameter function. It also suggests basis-optimization parameters from other geometric measurements, using settings chosen for the meshes in this study. This selection improved automation of cable model fitting for toric spines. However, a scalar function of the shape diameter is not a very robust way to choose ℓ, and better parameter heuristics remain an open problem.

The basis graph optimization process described in Section 2 is under active development and, as such, the software implementation has some extended functionality that is beyond the scope of this paper. While the routine is designed to converge to a stationary configuration, it is challenging to design a robust, efficient, and convergent algorithm that centers vertices without sometimes introducing other errors. As such, the implementation described in this paper is often best used for making small global improvements to the morphology graph in a limited number of iterations (10–20).

The overlap corrections used in geometric validation (Section 3.1.1) are local heuristics based on equal-radius, right-angle cylinder intersections, half of a Steinmetz solid per extra branch, not exact Boolean unions of the frustum segments. Junctions in reconstructed morphologies are in general neither equal-radius nor orthogonal. The volume and area scale factors (defaults 8/3 and 4) are therefore exposed as parameters in the `MASCAF` validation functions. While this is more accurate than a simple spherical-overlap model, these default values could be tuned more carefully.

An opportunity for future development of `MASCAF` is to extend the current MCFS approach with medial-axis centering methods, such as those used by TEASAR (Sato et al. 2000) and CoverageAxis (Wang et al. 2024), which are commonly used for skeletonization of 3D voxel data. While MCFS retains structural cycles, a medial-axis approach to basis graph refinement could potentially reduce errors in medial centering.

Lastly, it is important to recognize the inherent limitations of cable models as geometric representations of complex 3D shapes. Surface concavity, for example in toric spine #67, reliably leads to stubborn geometric errors, as shown in Table 1. While an accurate skeleton calculation should in principle preserve path lengths through the mesh volume, `MASCAF`'s cable fitting algorithm does not explicitly enforce this. Development of more flexible geometric representations of morphology for biophysical simulators would be a valuable contribution to this field of research.

4.2 Future Work

The development of `MASCAF` removes a key technical barrier in the detailed analysis of auditory space-specific neurons in the midbrain of the barn owl. High-resolution imaging data has provided unprecedented levels of detail in the anatomy and morphology of these systems, but construction of morphologically accurate simulations from that data has proven challenging. With cable-model construction from complex surface meshes now feasible, the next step involves constructing explanatory and predictive models for biophysical function of toric spines and SSNs. With `MASCAF`, we are able to perform simulations in `Arbor` of isolated toric spines with realistic synaptic and current mechanisms, probing the dynamics of signal integration. For the full SSN model, we take a multi-scale approach, grafting cable models of individual toric spines onto cable models of whole-neuron membranes. In this way we may reconstruct large and small scales at high resolution, and then probe full-scale simulations to illuminate the function of toric spines as components of a sophisticated neural computation system.

Declarations, Code, & Data

Research reported in this publication was supported by the National Institute of Neurological Disorders and Stroke and the National Institute on Deafness and Other Communication Disorders of the National Institutes of Health under award numbers RF1NS132812 and R01DC007690. The content is solely the responsibility of the authors and does not necessarily represent the official views of the National Institutes of Health.

The `MASCAF` Python code is free and open-source: https://github.com/jmrfox/mascaf. The nine toric spine surface meshes originally prepared by Sanculi et al. (2020) and used in this study are available at https://github.com/jmrfox/mascaf/tree/main/data/mesh/processed.

Appendix A MASCAF Pipeline Flowchart

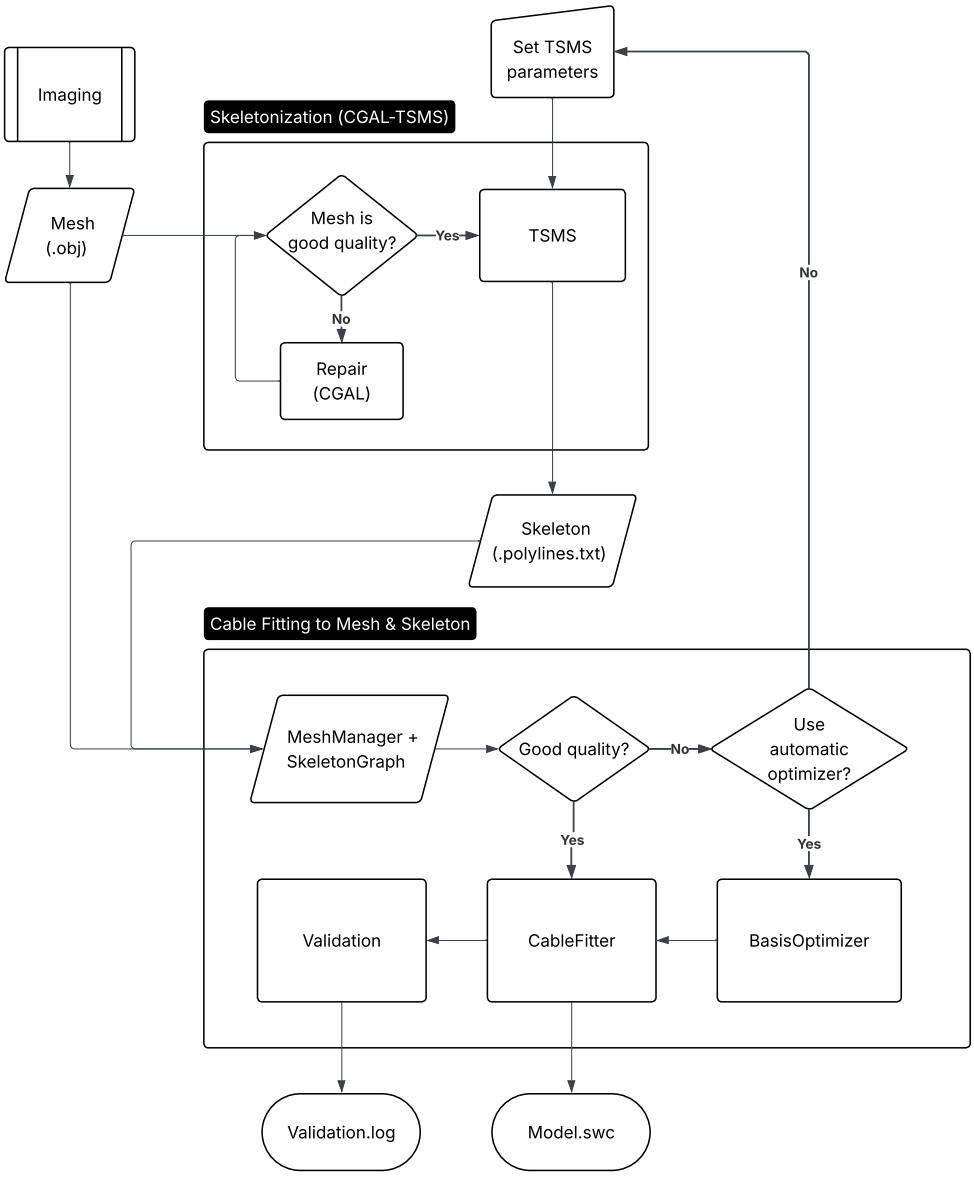


Fig. A1: The MASCAF pipeline.

Appendix B Algorithms

B.1 Basis Optimization

Algorithm 1 Basis Optimization

Require: Morphology graph G_M, surface mesh M_S; parameters γ, h, α_s, T, and centering-force threshold

Ensure: Optimized morphology graph G_M^*

$G_M^* \leftarrow G_M$
if pruning enabled **then**
  Delete short terminal branches of G_M^* below length threshold
end if
if snapping enabled **then**
  for all $\mathbf{v}_i \in G_M^*$ outside M_S **do**
    $\hat{\mathbf{u}}_i \leftarrow$ unit direction from $\mathbf{v}_i$ toward nearest surface point
    Find ordered ray–mesh hits $\{\mathbf{q}_j\}$ along $\hat{\mathbf{u}}_i$ (perturb if needed)
    Require even hit count and chord $\|\mathbf{q}_2 - \mathbf{q}_1\|$ above threshold
    $\mathbf{v}_i \leftarrow (\gamma - 1)\mathbf{q}_1 + \gamma \mathbf{q}_2$ ▷ default $\gamma \approx 1/4$
  end for
end if
if forcing enabled **then**
  $P \leftarrow$ protected nodes (terminals; optionally branch nodes)
  for $t = 1, \ldots, T$ **do**
    for all $\mathbf{v}_i \in G_M^* \setminus P$ **do**
      Compute centering force $\mathbf{f}_c$ and smoothing force $\mathbf{f}_s$ at $\mathbf{v}_i$
      $\mathbf{f} \leftarrow h\big[(1 - \alpha_s)\mathbf{f}_c + \alpha_s \mathbf{f}_s\big]$
      $f_{\max} \leftarrow$ tunable constant fraction (default 1/2) of distance from $\mathbf{v}_i$ to M_S along $\mathbf{f}$
      $\Delta \mathbf{v}_i \leftarrow \dfrac{\mathbf{f}}{\|\mathbf{f}\|} \min\big(\|\mathbf{f}\|, f_{\max}\big)$
      $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta \mathbf{v}_i$
    end for
    if $\|\mathbf{f}_c\|^2$ below threshold for all free nodes **then break**
    end if
  end for
end if
return G_M^*

B.2 Cable Model Fitting

Algorithm 2 Morphology Graph Construction and Radius Fitting

Require: Surface mesh M_S, skeleton graph G_Σ, max edge length ℓ

Ensure: Morphology graph G_M with local radii; subsequent stages noted below

$G_M \leftarrow \emptyset$
$\{g_\Sigma^k\} \leftarrow$ maximal non-branching sections of G_Σ ▷ paths and cycles
for all sections g_Σ^k **do**
  Resample g_Σ^k with spacing $\leq \ell$, preserving endpoints
  Add resampled nodes and edges to G_M
  if g_Σ^k is a cycle **then** retain cyclic connectivity in G_M
  end if
end for
Optionally run Basis Optimization (Algorithm above) on G_M
for all $\mathbf{v}_i \in G_M$ **do**
  for all incident edges e_{ij} **do**
    Intersect M_S with plane through $\mathbf{v}_i$ normal to $\mathbf{e}_{ij}$
    $r_{ij} \leftarrow \sqrt{A/\pi}$
  end for
  $r_i \leftarrow f(\{r_{ij}\})$ ▷ mean or median
end for
Globally scale radii so summed lateral surface areas match mesh surface area
For SWC export: break cycles by duplicating degree-2 vertices and write reconnection metadata
return G_M

Appendix C Related Software Tools

Fitting cable models to surface meshes is a common task in neuroscience, and there are a number of software tools that can be used to do this.

Commercial platforms such as `Imaris` (Oxford Instruments) provide powerful workflows for fitting cable models directly to imaging data and are widely used to reconstruct neuronal morphologies and connectomes from microscopy data. These tools are primarily designed for voxel-based image data and assume tree-like structures.

`Vaa3D` (Peng et al. 2014) is a multi-feature open-source program for imaging data manipulation that supports neuron reconstruction from volumetric datasets and provides cable-model outputs compatible with standard simulators. It is optimized for tracing neuronal arbors from image stacks and large-scale connectomics datasets.

Voxel-based medial-axis methods, most notably the Tree-structure Extraction Algorithm for Accurate and Robust Skeletons (`TEASAR`) (Sato et al. 2000), are related to the skeletonization approach presented here. They operate on labeled volumetric segmentations and iteratively extract centerlines by computing shortest paths to the surface in a grid. Modern implementations such as `Kimimaro` (Silversmith et al. 2021) extend this approach to large-scale dense connectomics datasets by incorporating efficient graph traversal and anisotropic distance transforms. `Kimimaro` is particularly designed to skeletonize large volumetric segmentations with high throughput and robustness. These methods are closely integrated with Google's `Neuroglancer` (Maitin-Shepard et al. 2021) ecosystem, including tools such as Igneous (Silversmith et al. 2022) for distributed data processing and `CloudVolume` (Tartavull and Silversmith 2019) for scalable data access and visualization. These methods operate on discrete voxel grids and produce skeletons that are inherently dependent on the resolution and discretization of the volumetric segmentation. While `TEASAR`-based methods can estimate local radius using distance-to-boundary fields, they do not directly fit cable models suitable for multicompartmental simulation.

The Coverage Axis approach (Wang et al. 2024), also medial-axis-based, provides an alternative strategy for extracting skeletal representations directly from surface meshes and point clouds. These methods aim to compute a centered and well-covered medial structure by combining geometric and sampling-based criteria, and have been shown to produce high-quality skeletons for complex shapes.

`Skeletor` (Schlegel 2021) is a popular tool for mesh skeletonization and can produce quality skeletons for typical tree-like geometries, but it failed for toric spines.

Importantly, although we focus on the `CGAL-TSMS` algorithm in this paper, `MASCAF` can accept any polyline-based skeleton representation as input, regardless of the method used to compute it. This modular design allows alternative skeletonization methods, including the medial-axis approaches listed above, to be incorporated into the `MASCAF` pipeline if desired, while retaining the advantages of robust radius estimation and simulator compatibility.

References

Abi Akar, N., Cumming, B., Karakasis, V., Küsters, A., Klijn, W., Peyser, A., Yates, S.: Arbor — A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: 2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP), pp. 274–282 (2019). https://doi.org/10.1109/EMPDP.2019.8671560

Alliez, P., Cohen-Steiner, D., Hemmer, M., Portaneri, C., Rouxel-Labbé, M.: 3D alpha wrapping. In: CGAL User and Reference Manual, 6.1.1 edn. (2026). CGAL Editorial Board. https://doc.cgal.org/6.1.1/Manual/packages.html#PkgAlphaWrap3

Ascoli, G.A., Donohue, D.E., Halavi, M.: Neuromorpho.org: A central resource for neuronal morphologies. The Journal of Neuroscience **27**(35), 9247–9251 (2007) https://doi.org/10.1523/JNEUROSCI.2055-07.2007

Agmon-Snir, H., Carr, C.E., Rinzel, J.: The role

of dendrites in auditory coincidence detection. Nature **393**(6682), 268–272 (1998) https://doi.org/10.1038/30505

Acciai, L., Soda, P., Iannello, G.: Automated neuron tracing methods: an updated account. Neuroinformatics **14**(4), 353–367 (2016) https://doi.org/10.1007/s12021-016-9310-0

Bower, J.M., Beeman, D.: The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System, 2nd edn. Springer, New York (1998). https://doi.org/10.1007/978-1-4612-1634-6

Brainard, M.S., Knudsen, E.I.: Experience-dependent plasticity in the inferior colliculus: a site for visual calibration of the neural representation of auditory space in the barn owl. The Journal of Neuroscience: The Official Journal of the Society for Neuroscience **13**(11), 4589–4608 (1993) https://doi.org/10.1523/JNEUROSCI.13-11-04589.1993

Blum, H.: A transformation for extracting new descriptors of shape. In: Wathen-Dunn, W. (ed.) Models for the Perception of Speech and Visual Form, pp. 362–380. MIT Press, Cambridge, MA (1967)

Carnevale, N.T., Hines, M.L.: The NEURON Book. Cambridge University Press, Cambridge (2006). https://doi.org/10.1017/CBO9780511541612

Cacciola, F., Rouxel-Labbé, M., Şenbaşlar, B., Komaromy, J.: Triangulated surface mesh simplification. In: CGAL User and Reference Manual, 6.1.1 edn. (2026). CGAL Editorial Board. https://doc.cgal.org/6.1.1/Manual/packages.html#PkgSurfaceMeshSimplification

Dawson-Haggerty et al.: trimesh. https://trimsh.org/. Version 3.2.0 (2019)

DeBello, W.M., Feldman, D.E., Knudsen, E.I.: Adaptive axonal remodeling in the midbrain auditory space map. Journal of Neuroscience **21**(9), 3161–3174 (2001) https://doi.org/10.1523/JNEUROSCI.21-09-03161.2001

Denk, W., Horstmann, H.: Serial block-face scanning electron microscopy to reconstruct three-dimensional tissue nanostructure. PLoS Biology **2**(11), 329 (2004) https://doi.org/10.1371/journal.pbio.0020329

Deistler, M., Kadhim, K.L., Pals, M., Beck, J., Huang, Z., Gloeckler, M., Lappalainen, J.K., Schröder, C., Berens, P., Gonçalves, P.J., Macke, J.H., *et al.*: Jaxley: differentiable simulation enables large-scale training of detailed biophysical models of neural dynamics. Nature Methods **22**, 2649–2657 (2025) https://doi.org/10.1038/s41592-025-02895-w

Dudley, F., Ray, S., Bhalla, U.S.: MOOSE: A parallel, hierarchical and flexibly defined multiscale simulation platform. Frontiers in Neuroinformatics **16**, 957347 (2022) https://doi.org/10.3389/fninf.2022.957347

Fischer, B.J., Anderson, C.H., Peña, J.L.: Multiplicative auditory spatial receptive fields created by a hierarchy of population codes. PLoS ONE **4**(11), 8015 (2009) https://doi.org/10.1371/journal.pone.0008015

Fox, J.: MASCAF. https://github.com/jmrfox/mascaf. Version 1.0.0 (2026)

Gold, J.I., Knudsen, E.I.: A site of auditory experience-dependent plasticity in the neural representation of auditory space in the barn owl's inferior colliculus. Journal of Neuroscience **20**(9), 3469–3486 (2000) https://doi.org/10.1523/JNEUROSCI.20-09-03469.2000

Gao, X., Loriot, S., Tagliasacchi, A.: Triangulated surface mesh skeletonization. In: CGAL User and Reference Manual, 6.1.1 edn. (2026). CGAL Editorial Board. https://doc.cgal.org/6.1.1/Manual/packages.html#PkgSurfaceMeshSkeletonization

Hering, H., Sheng, M.: Dendritic spines: structure, dynamics and regulation. Nature Reviews Neuroscience **2**(12), 880–888 (2001) https://doi.org/10.1038/35104061

Hagberg, A.A., Schult, D.A., Swart, P.J.: Exploring network structure, dynamics, and function using NetworkX. In: Varoquaux, G., Vaught,

T., Millman, J. (eds.) Proceedings of the 7th Python in Science Conference, Pasadena, CA, USA, pp. 11–15 (2008). https://doi.org/10.25080/TCWV9851

Hell, S.W., Wichmann, J.: Breaking the diffraction resolution limit by stimulated emission: stimulated-emission-depletion fluorescence microscopy. Optics letters **19**(11), 780–782 (1994) https://doi.org/10.1364/OL.19.000780

Knudsen, E.I., Brainard, M.S.: Visual instruction of the neural map of auditory space in the developing optic tectum. Science **253**(5015), 85–87 (1991) https://doi.org/10.1126/science.2063209

Koch, C., Jones, A.: Big science, team science, and open science for neuroscience. Neuron **92**(3), 612–616 (2016) https://doi.org/10.1016/j.neuron.2016.10.019

Knudsen, E.I., Konishi, M.: A neural map of auditory space in the owl. Science **200**(4340), 795–797 (1978) https://doi.org/10.1126/science.644324

Knudsen, E.I., Konishi, M.: Mechanisms of sound localization in the barn owl (tyto alba). Journal of comparative physiology **133**(1), 13–21 (1979) https://doi.org/10.1007/BF00663106

Knudsen, E.I., Knudsen, P.F.: Visuomotor adaptation to displacing prisms by adult and baby barn owls. Journal of Neuroscience **9**(9), 3297–3305 (1989) https://doi.org/10.1523/JNEUROSCI.09-09-03297.1989

Knudsen, E.I.: Auditory and visual maps of space in the optic tectum of the owl. Journal of Neuroscience **2**(9), 1177–1194 (1982) https://doi.org/10.1523/JNEUROSCI.02-09-01177.1982

Knudsen, E.I.: Instructed learning in the auditory localization pathway of the barn owl. Nature **417**(6886), 322–328 (2002) https://doi.org/10.1038/417322a

Koch, C.: Biophysics of Computation: Information Processing in Single Neurons. Oxford University Press, New York (1999). https://doi.org/10.1093/oso/9780195104912.001.0001

Konishi, M.: How the owl tracks its prey. American Scientist **61**(4), 414–424 (1973)

Keller, C.H., Takahashi, T.T.: Localization and identification of concurrent sounds in the barn owl. Journal of Neuroscience **25**(45), 10446–10461 (2005) https://doi.org/10.1523/JNEUROSCI.2093-05.2005

London, M., Häusser, M.: Dendritic computation. Annual Review of Neuroscience **28**, 503–532 (2005) https://doi.org/10.1146/annurev.neuro.28.061604.135703

Liu, S., Zhang, D., Liu, S., Feng, D., Peng, H., Cai, W.: Rivulet: 3d neuron morphology tracing with iterative back-tracking. Neuroinformatics **14**(4), 387–401 (2016) https://doi.org/10.1007/s12021-016-9302-0

Moiseff, A., Konishi, M.: Neuronal and behavioral sensitivity to binaural time differences in the owl. Journal of Neuroscience **1**(1), 40–48 (1981) https://doi.org/10.1523/JNEUROSCI.01-01-00040.1981

Mehta, K., Ljungquist, B., Ogden, J., Nanda, S., Ascoli, R.G., Ng, L., Ascoli, G.A.: Online conversion of reconstructed neural morphologies into standardized swc format. Nature Communications **14**(1), 7429 (2023) https://doi.org/10.1038/s41467-023-42931-x

Mainen, Z.F., Sejnowski, T.J.: Influence of dendritic structure on firing pattern in model neocortical neurons. Nature **382**(6589), 363–366 (1996) https://doi.org/10.1038/382363a0

Maitin-Shepard, J., Baden, A., Silversmith, W., Perlman, E., Collman, F., Blakely, T., Funke, J., Jordan, C., Falk, B., Kemnitz, N., tingzhao, Roat, C., Castro, M., Jagannathan, S., moenigin, Clements, J., Hoag, A., Katz, B., Parsons, D., Wu, J., Kamentsky, L., Chervakov, P., Hubbard, P., Berg, S., Hoffer, J., Halageri, A., Machacek, C., Mader, K., Roeder, L., Li, P.H.: google/neuroglancer:. Zenodo (2021). https://doi.org/10.5281/zenodo.5573294 . https://zenodo.org/records/5573294

Nimchinsky, E.A., Sabatini, B.L., Svoboda, K.: Structure and function of dendritic spines.

Annual Review of Physiology **64**, 313–353 (2002) https://doi.org/10.1146/annurev.physiol.64.081501.160008

Ohori, K.A.: 2D polygon repair. In: CGAL User and Reference Manual, 6.1.1 edn. (2026). CGAL Editorial Board. https://doc.cgal.org/6.1.1/Manual/packages.html#PkgPolygonRepair

Oxford Instruments: Imaris, Microscopy Image Analysis Software, 11 edn. Abingdon, UK. https://imaris.oxinst.com/

Payne, R.S.: Acoustic location of prey by barn owls (tyto alba). Journal of Experimental Biology **54**, 535–573 (1971) https://doi.org/10.1242/jeb.54.3.535

Peng, H., Bria, A., Zhou, Z., Iannello, G., Long, F.: Vaa3d enables real-time 3d visualization and quantitative analysis of large-scale biological image data sets. Nature Biotechnology **32**(4), 348–353 (2014) https://doi.org/10.1038/nbt.1612

Peña, J.L., DeBello, W.M.: Auditory processing, plasticity, and learning in the barn owl. ILAR journal **51**(4), 338–352 (2010) https://doi.org/10.1093/ilar.51.4.338

Peña, J.L., Konishi, M.: Auditory spatial receptive fields created by multiplication. Science **292**(5515), 249–252 (2001) https://doi.org/10.1126/science.1059201

Peña, J.L., Konishi, M.: From postsynaptic potentials to spikes in the genesis of auditory spatial receptive fields. Journal of Neuroscience **22**(13), 5652–5658 (2002) https://doi.org/10.1523/JNEUROSCI.22-13-05652.2002

Peng, H., Ruan, Z., Long, F., Simpson, J.H., Myers, E.W.: Virtual finger boosts three-dimensional imaging and microsurgery as well as terabyte volume image visualization and analysis. Nature Communications **5**, 4342 (2014) https://doi.org/10.1038/ncomms5342

Panagiotou, S., Sidiropoulos, H., Soudris, D., Negrello, M., Strydis, S.: EDEN: A high-performance, general-purpose, NeuroML-based neural simulator. Frontiers in Neuroinformatics **16**, 724336 (2022) https://doi.org/10.3389/fninf.2022.724336

Rall, W.: Branching dendritic trees and motoneuron membrane resistivity. Experimental Neurology **1**(5), 491–527 (1959) https://doi.org/10.1016/0014-4886(59)90046-9

Rall, W.: Theory of physiological properties of dendrites. Annals of the New York Academy of Sciences **96**(4), 1071–1092 (1962) https://doi.org/10.1111/j.1749-6632.1962.tb54120.x

Sato, M., Bitter, I., Bender, M.A., Kaufman, A.E., Nakajima, M.: TEASAR: Tree-structure extraction algorithm for accurate and robust skeletons. In: Proceedings of the 8th Pacific Conference on Computer Graphics and Applications, pp. 281–287 (2000). https://doi.org/10.1109/PCCGA.2000.883951 . IEEE Computer Society

Stimberg, M., Brette, R., Goodman, D.F.M.: Brian 2, an intuitive and efficient neural simulator. eLife **8**, 47314 (2019) https://doi.org/10.7554/eLife.47314

Silversmith, W., Bae, J.A., Li, P.H., Wilson, A.M.: Kimimaro: Skeletonize densely labeled 3D image segmentations. Version 3.0.0 (2021). https://doi.org/10.5281/zenodo.5539913

Schlegel, P.: Skeletor. https://github.com/navis-org/skeletor (2021)

Singheiser, M., Gutfreund, Y., Wagner, H.: The representation of sound localization cues in the barn owl's inferior colliculus. Frontiers in Neural Circuits **6**, 45 (2012) https://doi.org/10.3389/fncir.2012.00045

Siddiqi, K., Pizer, S.M. (eds.): Medial Representations: Mathematics, Algorithms and Applications. Computational Imaging and Vision, vol. 37. Springer, ??? (2008). https://doi.org/10.1007/978-1-4020-8658-8

Sanculi, D., Pannoni, K.E., Bushong, E.A., Crump, M., Sung, M., Popat, V., Zaher, C., Hicks, E., Song, A., Mofakham, N., Li, P., Antzoulatos, E.G., Fioravante, D., Ellisman, M.H., DeBello, W.M.: Toric spines at a site of

learning. eNeuro **7**(1), 0197–192019 (2020) https://doi.org/10.1523/ENEURO.0197-19.2019

Stuart, G., Spruston, N., Häusser, M.: Dendrites. Oxford University Press (2016) https://doi.org/10.1093/acprof:oso/9780198745273.001.0001

Spezio, M.L., Takahashi, T.T.: Frequency-specific interaural level difference tuning predicts spatial response patterns of space-specific neurons in the barn owl inferior colliculus. Journal of Neuroscience **23**(11), 4677–4688 (2003) https://doi.org/10.1523/JNEUROSCI.23-11-04677.2003

Silversmith, W., Zlateski, A., Bae, J.A., Tartavull, I., Kemnitz, N., Wu, J., Seung, H.S.: Igneous: Distributed dense 3d segmentation meshing, neuron skeletonization, and hierarchical downsampling. Frontiers in Neural Circuits **16** (2022) https://doi.org/10.3389/fncir.2022.977700

Tagliasacchi, A., Alhashim, I., Olson, M., Zhang, H.: Mean Curvature Skeletons. Computer Graphics Forum (2012) https://doi.org/10.1111/j.1467-8659.2012.03178.x

The CGAL Project: CGAL User and Reference Manual, 6.1.1 edn. (2026). CGAL Editorial Board. https://doc.cgal.org/6.1.1/Manual/packages.html

Tecuatl, C., Ljungquist, B., Ascoli, G.A.: Accelerating the continuous community sharing of digital neuromorphology data. FASEB BioAdvances **6**(7), 207 (2024) https://doi.org/10.1096/fba.2024-00048

Tartavull, I., Silversmith, W.: Cloudvolume. GitHub (2019)

Wavefront Technologies: Wavefront OBJ File Format. Alias/Wavefront, (1990). Alias/Wavefront. Technical Specification

Wang, Z., Dou, Z., Xu, R., Lin, C., Liu, Y., Long, X., Xin, S., Komura, T., Yuan, X., Wang, W.: Coverage Axis++: Efficient Inner Point Selection for 3D Shape Skeletonization (2024). https://doi.org/10.48550/arXiv.2401.12946 . https://arxiv.org/abs/2401.12946

Wang, D., Magee, L., Huo, B.-X., Banerjee, S., Li, X., Jayakumar, J., Lin, M.K., Ram, K., Wang, S., Wang, Y., Mitra, P.P.: Detection and skeletonization of single neurons and tracer injections using topological methods. bioRxiv (2020) https://doi.org/10.1101/2020.03.21.000323